

## Ancient Text (text)

Ancient viking people carved stories on stone using their own symbols. Björn found  $N$  versions of the same story and he already converted them to texts containing only the lowercase letters **a–z**. The texts are indexed from 0 to  $N - 1$ , let's denote them with  $S_0, S_1, \dots, S_{N-1}$ . All the texts have the same length  $K$ , but they might differ on some positions.

Now Björn wants to know which is the original version of the story, because he thinks that the others are just mutated copies of it. He suspects that the original text is the one that has the minimal average distance from all the others. We define the distance between two texts as the number of positions where they differ.



Figure 1: Ancient viking story on a stone.

More formally, the distance between two texts  $S_i$  and  $S_j$  denoted by  $dist(S_i, S_j)$  is the number of different  $k$  indices, for which  $0 \leq k \leq K - 1$  and  $S_i[k] \neq S_j[k]$ . We are looking for the text  $S_{orig}$  for which the value

$$avgdist(S_i) = \frac{1}{N-1} \sum_{j=0}^{N-1} dist(S_i, S_j)$$

is minimal. (Note that  $dist(S_i, S_i) = 0$ ). If there are multiple texts with the same average distance from the others, you should choose the one which has the smallest index of those.

Can you help Björn to determine which one is the original text?

📎 Among the attachments of this task you may find a template file `text.*` with a sample incomplete implementation.

## Input

The first line contains two integers  $N$  and  $K$ . The following  $N$  lines each contain a string  $S_i$  of length  $K$  containing only lowercase English letters “abc...z”.

## Output

You need to write a single line with an integer between 0 and  $N - 1$ , the index of the original text. If there are multiple possible solutions, print the one with the smallest index.

## Constraints

- $1 \leq N \leq 100\,000$ .
- $1 \leq K \leq 100\,000$ .
- $1 \leq N \cdot K \leq 1\,000\,000$ .

## Scoring

Your program will be tested against several test cases grouped in subtasks. In order to obtain the score of a subtask, your program needs to correctly solve all of its test cases.

- Subtask 1 (0 points)      Examples.
- Subtask 2 (6 points)       $N = 2$ .
- Subtask 3 (13 points)       $N = 3$ .
- Subtask 4 (23 points)       $1 \leq N, K \leq 100$ .
- Subtask 5 (27 points)       $S_i$  contains only a and b letters for each  $i = 0 \dots N - 1$ .
- Subtask 6 (31 points)      No additional limitations.

## Examples

| input                                                          | output |
|----------------------------------------------------------------|--------|
| 3 3<br>aab<br>aba<br>aaa                                       | 2      |
| 5 7<br>abcdefg<br>abcdefgh<br>abcdefgh<br>abcdef i<br>abcdef j | 1      |

## Explanation

In the **first sample case**,

- $avgdist(\mathbf{aab}) = \frac{1}{2}(dist(\mathbf{aab}, \mathbf{aba}) + dist(\mathbf{aab}, \mathbf{aaa})) = \frac{1}{2}(2 + 1) = 1.5$
- $avgdist(\mathbf{aba}) = \frac{1}{2}(dist(\mathbf{aba}, \mathbf{aab}) + dist(\mathbf{aba}, \mathbf{aaa})) = \frac{1}{2}(2 + 1) = 1.5$
- $avgdist(\mathbf{aaa}) = \frac{1}{2}(dist(\mathbf{aaa}, \mathbf{aab}) + dist(\mathbf{aaa}, \mathbf{aba})) = \frac{1}{2}(1 + 1) = 1$

So, the last text has the minimal average distance from the others, hence the answer is its index, which is 2.

In the **second sample case**, `abcdefh` has an average distance of 0.75 while the other texts have an average distance of 1, so the solution is the index of the first occurrence of `abcdefh`, which is 1.